

Employee Table Schema with Data Integrity Rules Part -2

Created By: Sharu latha. B

Course: AI-Driven Data Analytics

Batch No: FNB16

Project Objectives

The objective of this project is to design and implement a relational database system for managing employee information. The project focuses on applying SQL constraints to ensure data integrity, consistency, and reliability across multiple tables.

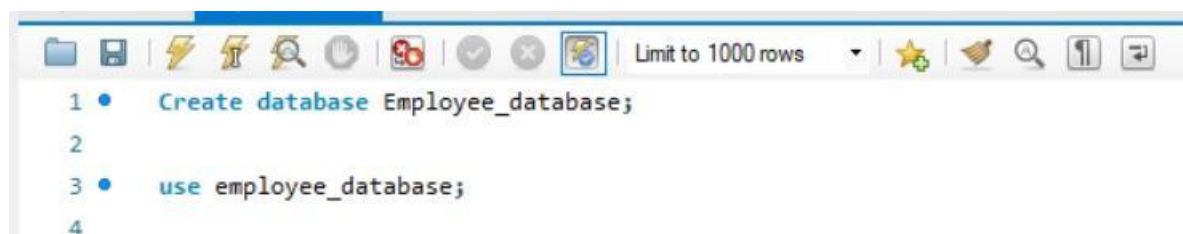
Data Insertion & Consistency

- Populate the Employees, Departments, and Location tables using given DML insert scripts.
- Ensure foreign key relationships are respected (department_id, location_id).

MySQL DML (Data Manipulation Language)

- Used to manipulate data inside tables (insert, update, delete, query).
- DISTINCT, WHERE, GROUP BY, JOIN, Window Functions.

Database and Table Creation (CREATE):



```
1 • Create database Employee_database;
2
3 • use employee_database;
4
```

Importance of Creating Tables Inside a Database Organized Data Storage

- ✦ Tables provide a structured way to store information in rows and

columns.

- ✦ Each table represents a specific entity (e.g. Employees, Departments, Locations).

Data Integrity

- ✦ Constraints (PRIMARY KEY, FOREIGN KEY, NOT NULL, CHECK) ensure that only valid data is stored.
- ✦ Prevents duplication and enforces rules automatically.

Table Creation (CREATE):

Example:

Table: Departments
Attributes:
• Department id – Primary Key
• department name

```

4
5  --- Create department table-
6
7  • Create table Departments(
8    department_id int auto_increment primary key,
9    department_name varchar(50) not null unique
10 );
11
12 • select * from departments;
  
```

Result Grid

department_id	department_name
NULL	NULL

Table: Location
Attributes:
✦ location id – Primary Key
✦ location name

```

13
14      --- create table Location-
15
16  • Create table Location(
17      location_id int auto_increment primary key,
18      location_name varchar(100) not null unique
19  );
20
21  • desc Location;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: IA

	Field	Type	Null	Key	Default	Extra
▶	location_id	int	NO	PRI	NULL	auto_increment
	location_name	varchar(100)	NO	UNI	NULL	

Table: Employees

Attributes:

- ✦ employee id – Primary Key
- ✦ Employee name
- ✦ Gender
- ✦ Age
- ✦ Hire date
- ✦ Designation
- ✦ Salary
- ✦ Department id – Foreign Key referencing department (department id)
- ✦ location id—Foreign Key referencing location (location id)

The screenshot shows a database management interface with a left-hand sidebar displaying a schema tree. The 'employee_database' is selected, showing tables like 'departments', 'employees', and 'location'. The main pane displays SQL code for creating the 'employees' table and a subsequent select statement. Below the code, the 'Table: employees' structure is detailed, listing columns and their data types. At the bottom, a 'Result Grid' shows a single row of null values for all columns.

Table: employees

Columns:

Column Name	Data Type
employee_id	int AI PK
department_id	int
location_id	int
employee_name	varchar(100)
gender	enum('M','F')
age	int
hire_date	date
designation	varchar(100)
salary	int

SQL Code:

```

25 • create table employees(
26     employee_id int auto_increment primary key,
27     department_id int,
28     location_id int,
29     employee_name varchar(100) not null,
30     gender enum('M','F'),
31     age int check (age >= 18),
32     hire_date date default (current_date),
33     designation varchar(100),
34     salary int,
35     foreign key (department_id) references departments (department_id),
36     foreign key(location_id) references location (location_id)
37 );
38
39 • select * from employees;
  
```

Result Grid:

employee_id	department_id	location_id	employee_name	gender	age	hire_date	designation	salary
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Clause & Operators- DISTINCT VALUES:

- Removes duplicate salary values.
- Shows only unique salaries. DML because it retrieves data.

The screenshot shows a SQL query being executed in a database management tool. The query is 'select distinct salary from employees;'. The result grid below shows the output, which is a list of unique salary values: 45000, 55000, 10000, and 15000.

SQL Code:

```

1 • use employee_database;
2
3 • select distinct salary from employees;
  
```

Result Grid:

salary
45000
55000
10000
15000

ALIAS (AS):

- Provide aliases for the "age" and "salary" columns as "Employee_Age" and "Employee_Salary", respectively.
 - Renames columns for readability.
 - Output will show Employee_Age and Employee_Salary. DML because it queries data.

```
18
19 • select age as employee_age , salary as employee_salary from employees;
```

	employee_age	employee_salary
▶	25	45000
	19	55000
	22	10000
	37	15000
	40	55000
	41	45000
	18	55000

WHERE CLAUSE & OPERATORS:

- Retrieve employees with a salary greater than ₹50000 and hired before 2016-01-01.
 - Filters employees with salary > 50,000 and hired before 2016.

```
31
32 -- WHERE CLAUSE & OPERATORS:Retrieve employees with a salary greater than ₹50000 and hired before 2016-01-01.
33
34 • select * from employees where salary > 50000 and date_of_joining < '2016-01-01';
```

	employee_id	department_id	location_id	employee_name	gender	date_of_joining	designation	salary	age
▶	15	5	5	Priya sharma	F	2015-01-01	Manager	55000	18
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

- Find the employee whose designation is missing and fill it with "Data Scientist".

```

46 • Update employees
47   Set designation = 'Data Scientist'
48   where designation is null;
49
50 • select * from employees;

```

Result Grid									
Filter Rows:									
Edit: Export/Import: Wrap Cell Content:									
	employee_id	department_id	location_id	employee_name	gender	date_of_joining	designation	salary	age
▶	1	1	1	Arun Kumar	M	2017-01-01	developer	45000	25
	2	2	2	Karthik	M	2016-01-01	Finance Officer	55000	19
	3	3	3	Rahul	M	2020-01-31	Intern	10000	22
	4	4	4	Meena	F	2020-01-31	Software Tester	15000	37
	6	4	4	Priya Sharma	F	2025-03-23	Manager	55000	40
	14	7	6	Priya	F	2020-01-31	Analyst	45000	41
	15	5	5	Priya sharma	F	2015-01-01	Manager	55000	18
	17	3	7	Meena	F	2017-01-01	Data Scientist	45000	21
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Sorting and Grouping Data:ORDER BY (DML)

- Find employees sorted by department ID in ascending order and salary in descending order.
 - Sorts employees by department ascending, salary descending.

```

59
60 • select * from employees order by department_id asc,salary desc;
61
62

```

Result Grid									
Filter Rows:									
Edit: Export/Import: Wrap Cell Content:									
	employee_id	department_id	location_id	employee_name	gender	date_of_joining	designation	salary	age
	1	1	1	Arun Kumar	M	2017-01-01	developer	45000	25
	2	2	2	Karthik	M	2016-01-01	Finance Officer	55000	19
	17	3	7	Meena	F	2017-01-01	Data Scientist	45000	21
	3	3	3	Rahul	M	2020-01-31	Intern	10000	22
	6	4	4	Priya Sharma	F	2025-03-23	Manager	55000	40
	4	4	4	Meena	F	2020-01-31	Software Tester	15000	37
	15	5	5	Priya sharma	F	2015-01-01	Manager	55000	18
	14	7	6	Priya	F	2020-01-31	Analyst	45000	41
	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

LIMIT:

- Display the first 5 employees hired in the year 2018.
 - Shows first 5 employees hired in 2018.

```

70
71 -- LIMIT: Display the first 5 employees hired in the year 2018.
72
73 • select * from employees
74 where Year(date_of_joining)= 2018
75 order by date_of_joining asc
76 limit 5;

```

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content: Fetch rows:									
	employee_id	department_id	location_id	employee_name	gender	date_of_joining	designation	salary	age
▶	5	2	5	surya	M	2018-01-01	IT	45000	22
	7	5	6	Jothi	F	2018-01-05	intern	20000	27
	8	7	2	Devi	F	2018-03-18	developer	30000	30
	9	1	1	Kumar	M	2018-05-01	Analyst	35000	30
	10	2	4	Dhinesh	M	2018-07-01	Software Tester	25000	25
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

AGGREGATE FUNCTIONS:

- **Calculate the sum of all salaries in the Finance department.**
 - Calculates total salary of Finance department.

```

83 • select sum(salary) Tot_Fin_salary
84   from employees
85   where department_id=2;
86
87
88

```

Result Grid		Filter Rows:	Export:	Wrap
	Tot_Fin_salary			
	125000			

```

82
83 • select sum(salary) Tot_Fin_salary
84   from employees
85   where department_id=2;
86
87 • select * from employees;
88 • select * from departments;

```

Result Grid		Filter Rows:	Edit
	department_id	department_name	
▶	7	Business Analyst	
	2	Finance	
	1	HR	
	3	IT	IT
	5	Research	
	4	Sales	
•	NULL	NULL	

- Find the minimum age among all employees. □ Finds youngest employee.
□ DML because it aggregates data.

```

--
90   -- Find the minimum age among all employees.
91
92 • select min(age) Min_age
93   from employees;
94

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	Min_age			
▶	18			

GROUP BY:

- **List the maximum salary for each location.**

- Finds highest salary per location.

```

94
95  -- GROUP BY:List the maximum salary for each location.
96
97 •  select location_id ,max(salary) max_salary
98     from employees
99     group by location_id;

```

Result Grid

	location_id	max_salary
▶	1	45000
	2	55000
	3	10000
	4	55000
	5	55000
	6	45000
	7	45000

- **Calculate the average salary for each designation containing the word 'Analyst'.**

```

100
101  --- Calculate the average salary for each designation containing the word 'Analyst'.
102
103 •  select designation , avg(salary) avg_salary
104     from employees
105     where designation like '%Analyst%'
106     group by designation;

```

Result Grid

	designation	avg_salary
▶	Analyst	40000.0000

HAVING:

Find departments with less than 3 employees.

- Shows departments with fewer than 3 employees. DML because it filters grouped data.

```

100
109    --- HAVING:Find departments with less than 3 employees.
110
111 •   select department_id, COUNT(*) AS total_employees
112     from employees
113     group by department_id
114     having COUNT(*) < 3;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	department_id	total_employees
▶	1	2
	3	2
	4	2
	5	2
	7	2

- Find locations with female employees whose average age is below 30. □ Shows locations where average female age < 30.

```

116    --- Find locations with female employees whose average age is below 30.
117
118 •   select location_id , avg(age) as avg_female_employee
119     from employees
120     where gender = 'F'
121     group by location_id
122     having avg(age) < 30;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	location_id	avg_female_employee
▶	5	18.0000
	7	21.0000

JOINS:

INNER JOIN:

- List employee names, their designations, and department names where employees are assigned to a department.

```

.24 --- Joins: INNER JOIN:
.25 --- List employee names, their designations, and department names where employees are assigned to a department.
.26
.27 • select e.employee_name,
.28       e.designation,
.29       d.department_name
.30 from employees e
.31 inner join departments d on e.department_id=d.department_id;
.32

```

employee_name	designation	department_name
Devi	developer	Business Analyst
Priya	Analyst	Business Analyst
Karthik	Finance Officer	Finance
surya	IT	Finance
Dhinesh	Software Tester	Finance
Arun Kumar	developer	HR
Kumar	Analyst	HR
Rahul	Intern	IT
Meena	Data Scientist	IT

LEFT JOIN:

- List all departments along with the total number of employees in each department, including departments with no employees.

□ Shows all departments, even if no employees

```

133 --- LEFT JOIN:
134 -- List all departments along with the total number of employees in each department, including departments with no employees.
135
136 • select d.department_id,
137       d.department_name,
138       count(e.employee_id) as Tot_employees_Each_department
139 from departments d
140 left join employees e
141 on d.department_id=e.department_id
142 group by d.department_id,d.department_name;
143

```

department_id	department_name	Tot_employees_Each_department
7	Business Analyst	2
2	Finance	3
1	HR	2
3	IT	2
5	Research	2
4	Sales	2

RIGHT JOIN:

- Display all locations along with the names of employees assigned to each location.
If no employees are assigned to a location, display NULL for employee name.

```

144 --- RIGHT JOIN:Display all locations along with the names of employees assigned to each location.
145 -- If no employees are assigned to a location, display NULL for employee name.
146
147 • use employee_database;
148
149 • select l.location_id,
150       l.location_name,
151       e.employee_name
152 from employees e
153 right join location l
154 on l.location_id = e.location_id;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	location_id	location_name	employee_name
▶	4	Bangalore	Meena
	4	Bangalore	Priya Sharma
	4	Bangalore	Dhinesh
	1	Chennai	Arun Kumar
	1	Chennai	Kumar
	2	Coimbatore	Karthik
	2	Coimbatore	Devi
	7	Gujarat	Meena

Result 1 x

CROSS JOIN

- Show all possible combinations of departments and locations.

```

156 --- CROSS JOIN- Show all possible combinations of departments and locations.
157
158 • select d.department_name,l.location_name
159 from departments d
160 cross join location l;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	department_name	location_name
▶	Sales	Bangalore
	Research	Bangalore
	IT	Bangalore
	HR	Bangalore
	Finance	Bangalore
	Business Analyst	Bangalore
	Sales	Chennai
	Research	Chennai

SELF JOIN:

- Show pairs of employees working in the same department, excluding self-pairs.

```

162
163 --- SELF JOIN: Show pairs of employees working in the same department, excluding self-pairs.
164
165 • select e1.employee_name as employee_1,
166         e2.employee_name as employee_2,
167         e1.department_id
168 from employees e1
169 join employees e2
170 on e1.department_id = e2.department_id
171 and e1.employee_id <> e2.employee_id;
172

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
employee_1	employee_2	department_id	
Arun Kumar	Kumar	1	
Karthik	surya	2	
Karthik	Dhinesh	2	
Rahul	Meena	3	
Meena	Priya Sharma	4	
surya	Karthik	2	
surya	Dhinesh	2	

Windows function

- Write a window function query to rank employees by salary using rank().

□ Ranks employees by salary overall.

```

174 --- Windows function-Write a window function query to rank employees by salary using rank().
175
176 • Select employee_name,
177         salary,
178         Rank() over (order by salary desc) as salary_rank
179 from employees;

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
employee_name	salary	salary_rank	
Karthik	55000	1	
Priya Sharma	55000	1	
Priya sharma	55000	1	
Arun Kumar	45000	4	
surya	45000	4	
Priya	45000	4	
Meena	45000	4	
Kumar	35000	8	

- Write a window function query to rank employees by salary within each department using DENSE_RANK()

```

2    --- Write a window function query to rank employees by salary within each department using DENSE_RANK()
3
4    • Select employee_name,
5            department_id,
6            salary,
7            dense_rank() over (partition by department_id order by salary desc) as dept_salary_rank
8    from employees;
9

```

employee_name	department_id	salary	dept_salary_rank
Arun Kumar	1	45000	1
Kumar	1	35000	2
Karthik	2	55000	1
surya	2	45000	2
Dhinesh	2	25000	3
Meena	3	45000	1
Rahul	3	10000	2
Priya Sharma	4	55000	1

- Write a window function query, Running total salary by department

```

190
191    --- Write a window function query, Running total salary by department
192
193    • select employee_name,
194            department_id,
195            salary,
196            SUM(salary) over (partition by department_id order by employee_id) as running_total_salary
197    from employees;
198

```

employee_name	department_id	salary	running_total_salary
Arun Kumar	1	45000	45000
Kumar	1	35000	80000
Karthik	2	55000	55000
surya	2	45000	100000
Dhinesh	2	25000	125000
Rahul	3	10000	10000
Meena	3	45000	55000
Meena	4	15000	15000

Conclusion

- DDL (Data Definition Language) was applied to create the database schema — defining tables (Departments, Location, Employees) and enforcing relationships through primary keys and foreign keys. This ensures structural integrity and consistency.
- DML (Data Manipulation Language) was used extensively to insert, update, and query data. Through clauses (DISTINCT, WHERE, ORDER BY, LIMIT), aggregate functions (SUM, MIN, MAX, AVG), grouping (GROUP BY, HAVING), and joins (INNER, LEFT, RIGHT, CROSS, SELF), we explored how to filter, sort, and combine data for meaningful insights.
- Window Functions (RANK, DENSE_RANK, SUM OVER) added advanced analytical capability, enabling ranking and running totals across partitions such as departments.
- This assignment strengthens the role of SQL as a powerful tool for a Data Analyst. By combining DDL for design and DML for operations, we achieve both data integrity and data intelligence — essential for decision-making in any organization.

Thank You

Entri Elevate